

Documentação Técnica Oficial

Arquitetura, tecnologias, dados, funcionalidades, segurança, integrações, backup, restauração, testes, publicação e operação.

VERSÃO	1.0
DATA	17/09/2026
STATUS	Aprovado para operação controlada
ESCOPO	Estado técnico consolidado da entrega

Documento canônico complementar ao STATUS_FINAL_DO_PROJETO.md. Não contém segredos ou credenciais reais.

Sumário

- 1. Propósito e escopo
- 2. Visão arquitetural
- 3. Tecnologias utilizadas
- 4. Organização do código
- 5. Modelo de dados
- 6. Migrations
- 7. API e middlewares
- 8. Funcionalidades por módulo
- 9. Frontend e experiência
- 10. Segurança
- 11. Variáveis de ambiente
- 12. Runtime e jobs
- 13. Testes e homologação
- 14. Desenvolvimento local
- 15. Build e publicação
- 16. Operação e monitoramento
- 17. Manutenção e governança
- 18. Parecer final

Versão 1.0 - 17/09/2026 Estado: sistema concluído, homologado e aprovado para operação controlada.

1. Propósito e escopo

O ACORDA RJ é uma plataforma web para coleta e administração de contatos comunitários, eventos, consentimentos, importações, relatórios, campanhas e mensageria oficial pelo WhatsApp Cloud API. O sistema inclui backup completo e restauração administrativa segura.

Esta é a documentação técnica oficial do estado entregue. Ela consolida as tecnologias, arquitetura, módulos, funcionalidades, segurança, dados, integrações, ambientes, testes, publicação e operação. Relatórios datados são evidências históricas; em caso de conflito sobre o estado atual, prevalecem este documento e [STATUS_FINAL_DO_PROJETO.md](#).

2. Visão arquitetural

```
Usuário público/admin
|
v
React 19 + Vite 8 (Vercel)
|
| HTTPS / JSON / multipart / Bearer JWT
v
Node.js 24 + Express 5 (DigitalOcean App Platform)
| |
| SQL parametrizado | HTTPS Graph API
v v
PostgreSQL 18 WhatsApp Cloud API
^ |
| | Webhook assinado
+----- HMAC / reconciliação
```

Princípios:

- backend é a autoridade de segurança, autorização e regras;
- frontend apresenta estados, mas não concede permissão;
- PostgreSQL é a fonte de verdade operacional;
- Meta é a fonte de verdade dos estados externos oficiais;
- operações críticas são persistentes e recuperáveis após restart;
- integrações externas são tratadas como falíveis e não transacionais.

3. Tecnologias utilizadas

3.1 Backend

Tecnologia	Versão/faixa	Uso
Node.js	24.x	Runtime do servidor e scripts.
JavaScript	CommonJS	Implementação do backend.
Express	5.1.x	API HTTP e middlewares.
PostgreSQL	18.x	Banco relacional e fonte operacional.
pg	8.16.x	Pool e SQL parametrizado, sem ORM.
bcrypt	6.x	Hash e verificação de senhas.
jsonwebtoken	9.x	Tokens JWT administrativos.
helmet	8.x	Headers HTTP de segurança.
cors	2.8.x	Restrição de origem do frontend.
compression	1.8.x	Compressão HTTP seletiva.
express-rate-limit	8.6.x	Limites globais e por fluxo.
dotenv	17.x	Configuração local por ambiente.
multer	2.4.x	Base do recebimento multipart controlado.
exceljs	4.4.x	Importação e exportação XLSX.

Ferramentas auxiliares: [pg_dump](#), [pg_restore](#), [psql](#), npm, PowerShell e scripts Node. O runtime de produção instala PostgreSQL client 18.4 pelo [backend/Aptfile](#).

3.2 Frontend

Tecnologia	Versão/faixa	Uso
React	19.2.x	Componentes e estado da interface.
React DOM	19.2.x	Renderização no navegador.
React Router DOM	7.18.x	Rotas públicas e administrativas.
Vite	8.1.x	Desenvolvimento e build.
@vitejs/plugin-react	6.x	Integração React/Vite.
qrcode.react	4.2.x	QR Codes de eventos.
Fetch API	nativa	Comunicação HTTP com a API.
CSS	nativo	Layout responsivo e identidade visual.

3.3 Infraestrutura, formatos e protocolos

- DigitalOcean App Platform para o backend;
- PostgreSQL 18 gerenciado com TLS;
- Vercel para o frontend SPA;

- HTTPS, JSON, multipart/form-data e CSV/XLSX/VCF;
- JWT Bearer, bcrypt, HMAC SHA-256 e SHA-256;
- WhatsApp Cloud API e Graph API da Meta;
- webhooks Meta assinados;
- dump PostgreSQL custom e pacote único `.acorda`;
- Git e npm lockfiles para versionamento e builds reproduzíveis.

4. Organização do código

4.1 Backend

```
backend/  
Aptfile  
database/  
  criar_banco.sql  
  migrations/001...023  
scripts/  
src/  
app.js  
server.js  
config/  
middlewares/  
modules/  
utils/
```

O fluxo predominante é:

```
route -> middleware -> controller -> service -> model -> PostgreSQL
```

- routes: contrato HTTP e middlewares do endpoint;
- controllers: entrada/saída HTTP;
- services: regras, transações e orquestração;
- models: SQL parametrizado e persistência;
- middlewares: autenticação, autorização, limites e erros;
- config: banco, ambiente e parâmetros compartilhados;
- scripts: migrations, administração e testes isolados.

4.2 Frontend

```
frontend/src/  
components/  
data/  
pages/  
services/  
styles/  
utils/  
App.jsx  
main.jsx
```

Páginas compõem jornadas; componentes reutilizam UI; services centralizam a API; utils tratam token/formatação; styles separam áreas pública, legal, administrativa, login e restauração.

5. Modelo de dados

O conjunto operacional possui 31 tabelas e 166 bairros iniciais. Os principais grupos são:

Domínio	Tabelas principais
Controle	<code>schema_migrations</code> , <code>configuracoes_sistema</code> , <code>historico_configuracoes_sistema</code>
Usuários	<code>usuarios</code> , <code>tentativas_login</code>
Contatos	<code>contatos</code> , <code>historico_contatos</code> , <code>origens</code> , <code>bairros</code>
Privacidade	<code>consentimentos</code> , <code>aceites_privacidade</code> , <code>textos_formulario</code> , <code>solicitacoes_exclusao</code>
Eventos	<code>eventos</code> , <code>contato_eventos</code> , <code>historico_eventos</code>
Importação	<code>importacoes</code> , <code>importacao_linhas</code>
Comunicação	<code>comunicacoes</code> , <code>historico_comunicacoes</code> , <code>numeros_whatsapp</code>
Campanhas	<code>campanhas</code> , <code>campanha_lotes</code> , <code>campanha_participacoes</code> , <code>campanha_tentativas</code>
Mensageria	<code>historico_status_mensageria</code> , <code>eventos_webhook_mensageria</code>
Templates/Meta	<code>modelos_mensagem</code> , <code>historico_modelos_mensagem_meta</code> , <code>sincronizacoes_limite_meta</code>
Backup	<code>backups_banco</code>

O schema separado `recuperacao` preserva controle de manutenção, `auth_epoch`, operações/auditoria de restore, webhooks recebidos durante manutenção e referência do pré-backup. Ele não contém dados de negócio duplicados nem FKs para o conjunto restaurado.

Garantias estruturais:

- telefone canônico único;
- contato único por campanha e por evento;
- PKs, FKs, `NOT NULL`, `CHECK` e índices;
- sequences consistentes com IDs existentes;
- triggers e funções para invariantes/auditoria;
- histórico preservado em vez de exclusão destrutiva;
- ledger de migration com checksum SHA-256 e advisory lock.

6. Migrations

As migrations são contínuas de 001 a 023:

Faixa	Evolução
001-005	Estrutura, nomes, eventos, múltiplos eventos e telefones.
006-007	Campanhas, lotes, mensageria e triggers.
008	Histórico inicial de backups.
009-015	Meta Cloud API, VCF, capacidade, webhooks e templates oficiais.
016-020	Status, arquivamento, unicidade, idade e exclusão lógica de modelos.
021	Resiliência de envio e webhook.
022	Metadados do backup completo.
023	Controle preservado de recuperação.

`database/criar_banco.sql` serve somente a banco vazio. Em banco existente, o único caminho é `npm run banco:migrar`. Migration aplicada nunca deve ser editada.

7. API e middlewares

Ordem relevante em `app.js`:

1. Helmet e CORS;
2. request ID, compression, rate limit e concorrência;
3. barreira persistente de manutenção;
4. webhook WhatsApp antes do parser JSON, para validar corpo bruto;
5. parser JSON limitado;
6. limitador específico do cadastro público;
7. saúde, público e autenticação;
8. cache privado desabilitado e JWT para `/api/admin/*`;
9. rotas administrativas;
10. 404 e tratamento central de erros.

Grupos de endpoint:

- `/api/saude/vivo` e `/api/saude/pronto`;
- `/api/publico/contatos`;
- `/api/autenticacao`;
- `/api/webhooks/whatsapp`;
- `/api/admin/contatos`, `/origens`, `/eventos` e `/importacoes`;
- `/api/admin/relatorios` e `/usuarios`;
- `/api/admin/solicitacoes-exclusao`;
- `/api/admin/campanhas` e `/mensageria`;
- `/api/admin/backups` e `/restauracoes`.

8. Funcionalidades por módulo

8.1 Autenticação e usuários

- login por e-mail e senha;
- bcrypt para hashes;
- JWT com expiração configurável;
- perfis `operador` e `administrador`;
- bloqueio temporário por excesso de falhas;
- proteção contra enumeração de e-mail;
- `auth_epoch` global para invalidar sessões;
- gestão administrativa de usuários ativos.

8.2 Contatos e origens

- cadastro público e manual;
- telefone canônico único;
- origem, bairro e dados de contato;
- busca, filtros, paginação e detalhes;
- histórico de alterações;

- normalização consistente de nomes e telefones;
- proteção contra duplicidade concorrente.

8.3 Privacidade, consentimentos e exclusões

- consentimentos separados para mensagem e ligação;
- versionamento de decisão, texto, origem e data;
- aceite de política de privacidade;
- revogação e bloqueio sem apagar histórico;
- páginas de privacidade, termos e direitos;
- solicitação, análise e execução administrativa de exclusão/anonimização;
- auditoria e verificação de dependências.

Nenhum histórico operacional expira por idade. Cleanup remove somente artefatos técnicos temporários.

8.4 Eventos

- vários eventos ativos;
- criação e gestão administrativa;
- participação pública por link;
- QR Code;
- vínculo único contato/evento;
- histórico de eventos e participantes.

8.5 Importações

- VCF, CSV e XLSX;
- validação, prévia e confirmação;
- normalização idêntica ao cadastro manual;
- rastreamento de linhas válidas, inválidas e duplicadas;
- persistência de importação e resultados;
- limites de arquivo e tratamento seguro.

8.6 Relatórios

- indicadores administrativos;
- filtros de consulta;
- exportação CSV e XLSX;
- limite configurável de registros;
- autorização e filtros revalidados no backend.

8.7 Campanhas

- criação e arquivamento com histórico;
- template e snapshot imutável dos filtros;
- elegibilidade por consentimento, bloqueio, exclusão, template e capacidade;
- lotes com chave idempotente e ordem;
- reservas transacionais;

- participação única por campanha/contato;
- tentativas múltiplas ligadas à mesma participação;
- visualização de estados, falhas e capacidade.

8.8 Templates Meta

- ID oficial, nome, idioma, categoria, status e componentes;
- submissão e sincronização administrativa;
- atualização por webhook;
- parâmetros e prévia, inclusive imagem;
- envio apenas quando oficialmente apto;
- exclusão manual/lógica protegida;
- preservação de histórico e vínculos.

8.9 Mensageria

- provider isolado para WhatsApp Cloud API;
- tentativas persistidas antes da chamada externa;
- estados [pendente](#), [enviando](#), [enviada](#), [entregue](#), [lida](#), [falhou](#);
- histórico de transições;
- idempotência, locks e proteção contra duplo clique;
- capacidade efetiva pelo menor limite interno/oficial;
- recuperação segura após restart;
- distinção entre falha explícita e resultado indeterminado;
- proibição de retry automático do indeterminado.

8.10 Webhook e opt-out

- GET de verificação da Meta;
- POST com HMAC SHA-256 sobre corpo bruto;
- comparação segura da assinatura;
- idempotência por evento;
- tratamento de duplicados e fora de ordem;
- preservação de evento antecipado/sem correlação;
- acompanhamento individual de payload misto;
- persistência fora do snapshot durante manutenção;
- opt-out que revoga mensagens, bloqueia contato e impede novas tentativas.

8.11 Backup

- arquivo único `acorda-rj-completo-AAAA-MM-DD_HH-mm-ss.acorda`;
- dump custom completo do PostgreSQL;
- estrutura, dados, sequences, constraints, índices, funções, triggers e large objects;
- exclusão explícita do schema [recuperacao](#);
- manifesto autenticado por HMAC;

- SHA-256, versão PostgreSQL e migrations;
- processamento por streaming;
- lock, timeout, fila e tamanho máximos;
- histórico administrativo e download privado único;
- retenção apenas do artefato técnico temporário.

O `.acorda` não é criptografado e contém dados sensíveis. Deve ser custodiado externamente. Backup/PITR do provedor permanece complementar.

8.12 Restauração

- upload por streaming e arquivo interno privado;
- guardas de tamanho, timeout, disco, memória, expansão e registro;
- autenticação e inspeção em banco isolado;
- compatibilidade de PostgreSQL/migrations;
- exigência de administrador ativo no candidato;
- confirmação administrativa forte;
- manutenção persistente e incremento de `auth_epoch`;
- drenagem comprovada de requisições e jobs;
- pré-backup estabilizado e custódia obrigatória;
- restore sem Meta e sem mensagens;
- validação pós-restore;
- reconciliação de webhooks preservados;
- novo login, conferência somente leitura e liberação manual;
- estado de recuperação necessária em qualquer falha crítica.

9. Frontend e experiência

Rotas públicas: `/participar`, `/privacidade`, `/termos`, `/excluir-dados` e `/login`.

Rotas autenticadas: dashboard, contatos, novo contato, detalhes, campanhas, importações, relatórios, eventos e ajuda. Rotas exclusivas de administrador: usuários, solicitações de exclusão e backups/restaurações.

Comportamentos técnicos:

- serviço HTTP central e Bearer Token;
- 401 encerra a sessão local;
- retry progressivo somente para GET transitório;
- mutações nunca são repetidas automaticamente;
- loading e botões desabilitados em ações críticas;
- mensagens seguras sem stack trace;
- layout responsivo;
- proteção visual de rotas sem substituir o backend;
- fluxo de restore em três momentos;
- comprovante de sucesso somente após restauração validada.

10. Segurança

Aplicação

- bcrypt e JWT expirável;
- `auth_epoch` preservado fora do snapshot;
- autorização por perfil no servidor;
- Helmet, CORS estrito, no-store, rate limit e concorrência;
- limites de JSON, upload e processamento;
- SQL parametrizado e transações;
- constraints, locks e idempotência;
- erros/logs sanitizados e request ID;
- HMAC de webhook e backup;
- arquivos privados e subprocessos sem shell;
- desligamento gracioso e timeouts.

Segredos

Banco, JWT, assinatura e Meta ficam apenas no backend/painel seguro. Arquivos `.env.example` guardam nomes, nunca valores reais. O frontend recebe somente configurações públicas. Rotação deve ocorrer pelo provedor e ser seguida de nova validação.

Limites conhecidos

- HMAC autentica; não criptografa o backup;
- serviço externo pode aceitar uma mensagem e a resposta se perder;
- teste local não reproduz integralmente o cgroup de produção;
- Meta, rede, Vercel, DigitalOcean e banco gerenciado podem falhar separadamente.

11. Variáveis de ambiente

Backend

Grupos documentados em `backend/.env.example`:

- runtime, porta e origem;
- banco, TLS, pool e timeouts;
- JWT e bloqueio de login;
- proxy, concorrência e rate limits;
- cache de bairros;
- `pg_dump`, backup e assinatura;
- `pg_restore`, upload e capacidade de restore;
- limite de relatórios;
- credenciais e parâmetros Meta/WhatsApp.

Produção exige JWT, origem HTTPS, banco TLS, chave de backup e integrações Meta essenciais. PostgreSQL remoto exige orçamento explícito em `RESTORE_BANCO_DISPONIVEL_BYTES` para habilitar restore.

Frontend

- `VITE_API_URL`;
- `VITE_WHATSAPP_NUMERO`;
- `VITE_PRIVACIDADE_EMAIL`.

12. Runtime e jobs

`server.js` valida o ambiente, abre HTTP, reconhece restore interrompido, inicia sincronização de templates e executa recuperação de mensageria sem sobreposição. No encerramento, para jobs, fecha conexões HTTP e encerra o pool.

Configurações relevantes incluem request timeout compatível com upload, headers timeout, keep-alive e limite por socket. A manutenção persistente bloqueia novas mutações enquanto permite o caminho administrativo de recuperação e o recebimento durável de webhooks válidos.

13. Testes e homologação

O backend possui runners específicos para banco, normalização, público, admin, cadastro manual, importações, relatórios, segurança, privacidade, eventos, exclusões, campanhas, templates, Meta simulada, limite Meta, webhook, backup, restore, upload, pacote `.acorda`, resiliência e E2E.

O frontend testa backup, restauração renderizada, prévia de modelo/imagem e build.

Homologação final registrada:

- cenário de 2.000 contatos e 511 verificações sem duplicidade observada;
- restart, concorrência e resultado indeterminado sem reenvio automático;
- 23 migrations e 31 tabelas verificadas;
- webhook/opt-out autenticado e idempotente;
- 41 verificações administrativas de backup e 20 de geração/falha;
- 112 verificações da Fase 2 e 46 de restauração completa;
- frontend, instalação limpa, segurança e dependências aprovados;
- RSS aproximado de 111,18 MiB na carga e 119,80 MiB no upload grande;
- `npm audit` sem vulnerabilidades no fechamento.

Os providers Meta foram mocks. Nenhuma mensagem real foi enviada pela homologação.

14. Desenvolvimento local

Banco vazio:

```
createdb criar_banco
psql --set ON_ERROR_STOP=1 --dbname criar_banco --file backend/database/criar_banco.sql
```

Backend:

```
cd backend
npm ci
Copy-Item .env.example .env
npm start
```

Frontend, em outro terminal:

```
cd frontend
npm ci
```

```
Copy-Item .env.example .env
npm run dev
```

Endereços padrão: frontend <http://localhost:5173>, API <http://localhost:3000>, liveness </api/saude/vivo> e readiness </api/saude/pronto>.

Nunca rode migrations/testes antes de confirmar que o banco é local ou descartável. Runners destrutivos devem recusar host não loopback.

15. Build e publicação

Backend - DigitalOcean

- componente com raiz `backend`;
- buildpack Node, sem Dockerfile;
- `Aptfile` para PostgreSQL client 18.4;
- `heroku-postbuild`: valida `pg_dump --version`;
- `prestart`: executa migrations;
- `start`: executa `node src/server.js`;
- variáveis no painel seguro;
- health checks de liveness/readiness;
- PostgreSQL 18 com TLS.

Frontend - Vercel

- raiz `frontend`;
- build `npm run build`;
- variáveis públicas da aplicação;
- `VITE_API_URL` apontando para HTTPS;
- `FRONTEND_URL` do backend igual à origem publicada;
- fallback SPA e rotas diretas configurados.

16. Operação e monitoramento

Monitore:

- liveness/readiness, reinícios e erros não tratados;
- RSS, heap, event-loop delay e pool;
- 429/503, timeouts, locks e falhas de banco;
- tentativas antigas em `enviando` e resultados indeterminados;
- webhooks pendentes/sem correlação e HMAC inválido;
- opt-outs, capacidade Meta e divergências de templates;
- espaço temporário, backups e restores;
- migrations e versões de `pg_dump/pg_restore`.

Smoke pós-deploy: saúde, login/perfis, cadastro controlado, consultas, eventos, importação, relatórios, abertura de campanhas sem envio, challenge do webhook e geração/download de backup. Não restaure produção como smoke test.

O primeiro envio real deve usar lote pequeno, monitoramento ativo e possibilidade de interrupção imediata.

17. Manutenção e governança

- escopo funcional está congelado;
- mudanças exigem autorização e regressão proporcional;
- correção deve partir de defeito reproduzido ou risco comprovado;
- migration aplicada não é reescrita;
- exclusão ampla, produção, Meta real e deploy exigem procedimento específico;
- documentos superados permanecem marcados como históricos;
- revisão estática, teste local, isolamento e produção são evidências distintas;
- `PROMPT_MESTRE.md` é o roteiro oficial de reconstrução do zero.

18. Parecer final

SISTEMA APROVADO PARA OPERAÇÃO CONTROLADA.

Essa aprovação descreve a versão e os testes registrados em 17/09/2026. Não é garantia absoluta contra indisponibilidade externa, mudança futura de API ou configuração incorreta de produção. Qualquer mudança posterior deve atualizar a documentação e repetir os testes pertinentes.